Documentation Android
Gestion du stockage des données

Mouammar Soule

10 Janvier 2026

1 Introduction

Le développement d'applications Android nécessite une gestion efficace, sécurisée et adaptée des données. Selon la nature des informations manipulées (paramètres, fichiers, médias ou données structurées), Android met à disposition plusieurs mécanismes de stockage.

Cette documentation présente une vue d'ensemble des principaux modes de stockage disponibles sous Android, leurs caractéristiques, ainsi que les cas d'utilisation recommandés, en s'appuyant sur la documentation officielle.

2 Panorama des modes de stockage sous Android

Android propose différents types de stockage, chacun répondant à des besoins spécifiques en matière de confidentialité, de persistance et de partage.

2.1 Panorama des modes de stockage (1/2)

Mode de stockage	**Visible par autres apps**	**Persiste après désinstallation**	**Types de données typiques**
App-specific (Interne)	Non	Non	Données privées, cache, fichiers temporaires
App-specific (Externe)	Non	Non	Fichiers volumineux privés
Shared storage – Media	Oui	Oui	Photos, vidéos, musiques
Shared storage – Documents	Oui	Oui	PDF, documents, téléchargements
Préférences	Non	Non	Paramètres utilisateur
Bases de données	Non	Non	Données structurées complexes

2.2 Panorama des modes de stockage (2/2)

Mode de stockage	API / Classes principales	Utilisation recommandée
App-specific (Interne)	`getFilesDir()`, `getCacheDir()`	Données sensibles
App-specific (Externe)	`getExternalFilesDir()`	Besoin de plus d'espace
Shared storage – Media	MediaStore API	Partage multimédia
Shared storage – Documents	SAF (`ACTION_OPEN_DOCUMENT`)	Fichiers partagés
Préférences	DataStore / SharedPreferences	Configuration
Bases de données	Room / SQLite	Données relationnelles

3 Stockage de données simples avec SharedPreferences

SharedPreferences permet de stocker de petites quantités de données sous forme de paires clé/valeur. Ce mécanisme est adapté aux paramètres utilisateur ou aux options de configuration.

3.1 Exemple d'implémentation

```
SharedPreferences prefs = getSharedPreferences(
    "app_prefs", Context.MODE_PRIVATE);

prefs.edit()
     .putString("username", "UtilisateurTest")
     .apply();

String value = prefs.getString("username", "Par défaut");
```

4 Stockage spécifique à l'application (App-specific Storage)

Le stockage spécifique à l'application (*App-specific Storage*) désigne des emplacements de stockage réservés exclusivement à une application Android. Les fichiers enregistrés dans ces répertoires ne sont pas accessibles aux autres applications et sont automatiquement supprimés lorsque l'utilisateur désinstalle l'application.

Android distingue deux formes de stockage spécifique à l'application : le stockage interne et le stockage externe spécifique.

4.1 Stockage interne

Le stockage interne correspond à un espace privé situé dans la mémoire interne de l'appareil. Il est totalement isolé des autres applications et ne nécessite aucune autorisation particulière pour la lecture ou l'écriture.

Ce type de stockage est particulièrement adapté pour :

- Les données sensibles ou confidentielles.
- Les fichiers nécessaires uniquement au fonctionnement interne de l'application.
- Les données de petite ou moyenne taille.

L'accès au répertoire interne se fait à l'aide de la méthode `getFilesDir()` fournie par le contexte de l'application.

```
File file = new File(getFilesDir(), "example.txt");
```

4.2 Écriture et lecture dans le stockage interne

L'écriture et la lecture de fichiers dans le stockage interne peuvent être réalisées à l'aide de flux (*streams*).
Le mode `Context.MODE`$_{PRIVATEgarantitquelefichierestaccessibleuniquementparl'applicationcourante.}$

```
// Écriture dans un fichier interne
FileOutputStream fos = openFileOutput(
    "example.txt", Context.MODE_PRIVATE);
fos.write("Hello Storage".getBytes());
fos.close();

// Lecture du fichier interne
FileInputStream fis = openFileInput("example.txt");
BufferedReader reader =
    new BufferedReader(new InputStreamReader(fis));
String line = reader.readLine();
fis.close();
```

4.3 Stockage externe spécifique à l'application

Le stockage externe spécifique à l'application est situé sur l'espace de stockage externe de l'appareil (mémoire partagée ou carte SD), mais dans un répertoire dédié à l'application. Bien que cet espace soit physiquement externe, les fichiers restent privés à l'application et sont également supprimés lors de la désinstallation.

Ce mode de stockage est recommandé lorsque :

- Les fichiers sont volumineux (images, journaux, exports).
- L'espace de stockage interne est insuffisant.
- Les données ne doivent pas être partagées avec d'autres applications.

Sur Android 4.4 et versions ultérieures (y compris Android 11+), l'accès à ce répertoire ne nécessite aucune permission de stockage.

```
File dir = getExternalFilesDir(null);
File file = new File(dir, "external.txt");
```

4.4 Synthèse

En résumé, le stockage interne doit être privilégié pour les données sensibles et critiques, tandis que le stockage externe spécifique à l'application est plus adapté aux fichiers volumineux ou nécessitant davantage d'espace, tout en conservant un niveau de confidentialité équivalent.

5 Stockage partagé : MediaStore

Le stockage partagé permet d'accéder à des fichiers multimédias visibles par l'utilisateur (images, vidéos, audio). Android fournit l'API MediaStore qui agit comme un index optimisé des contenus multimédias.

5.1 Exemple d'accès aux images

```
String[] projection = {
    MediaStore.Images.Media._ID,
    MediaStore.Images.Media.DISPLAY_NAME
};

Cursor cursor = getContentResolver().query(
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
    projection, null, null, null
);
```

6 Base de données locale avec Room

Room est une bibliothèque Jetpack fournissant une abstraction au-dessus de SQLite. Elle facilite la persistance de données structurées et garantit la validité des requêtes SQL à la compilation.

6.1 Principe

Room repose sur :

- Une entité représentant une table
- Un DAO définissant les opérations
- Une base de données centrale

```
@Database(entities = {User.class}, version = 1)
public abstract class AppDatabase
        extends RoomDatabase {
    public abstract UserDao userDao();
}
```

7 Content Providers

Les Content Providers permettent de partager des données entre applications via des URI de type content://. Ils sont utilisés par Android pour exposer des données système telles que les contacts ou le calendrier.

7.1 Principe général

```
public class ExampleProvider
        extends ContentProvider {

    @Override
    public boolean onCreate() { return true; }

    @Override
    public Cursor query(...) {
        return cursorResult;
    }
}
```

8 Conclusion

Android met à disposition plusieurs mécanismes de stockage adaptés à des besoins variés. Le choix du mode de stockage dépend principalement de la nature des données, de leur persistance et de leur niveau de partage.

Dans cette documentation, nous avons exploré les principales solutions proposées par Android, depuis le stockage simple avec SharedPreferences jusqu'aux bases de données locales avec Room et au partage inter-applications via les Content Providers. Une bonne compréhension de ces mécanismes est essentielle pour concevoir des applications Android robustes, sécurisées et performantes.

9 Liens et ressources

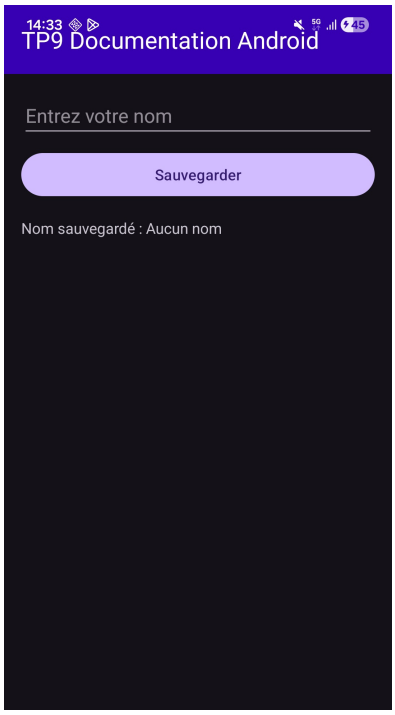
- SharedPreferences : https://developer.android.com/training/data-storage/shared-preferences
- App-specific storage : https://developer.android.com/training/data-storage/app-specific
- Shared storage : https://developer.android.com/training/data-storage/shared
- Room / SQLite : https://developer.android.com/training/data-storage/room
- Content Providers : https://developer.android.com/guide/topics/providers/content-providers

Outils utilisés

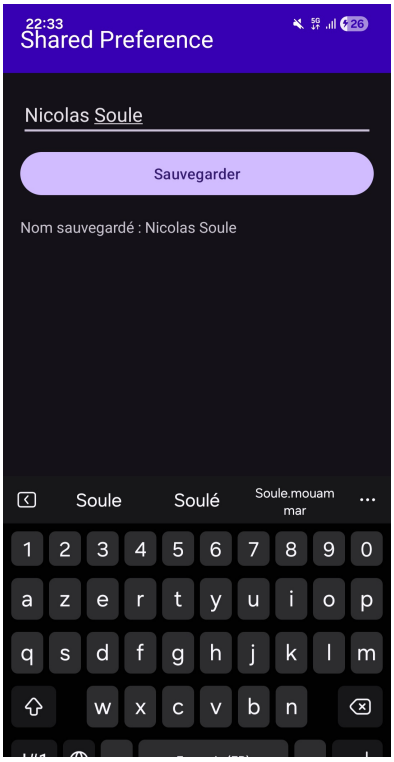
- Documentation Android officielle
- Grok (xAI ancien Twitter) : pour le Tableau en début de présentation et les corréctions d'orthographe pour la rédaction.

10 Captures d'écran — Démonstrations

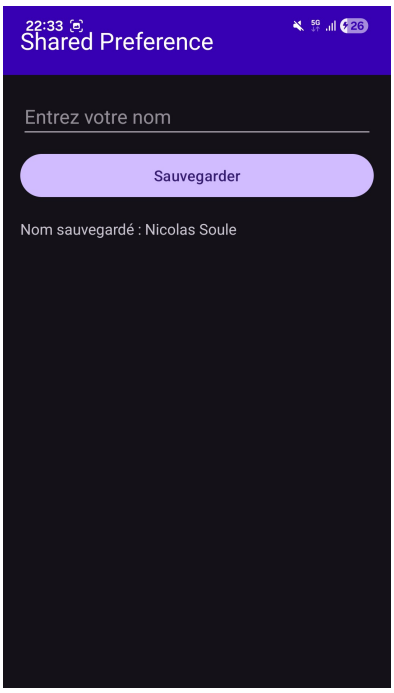
10.1 Démonstration : SharedPreferences



(a) (a) Écran principal des préférences



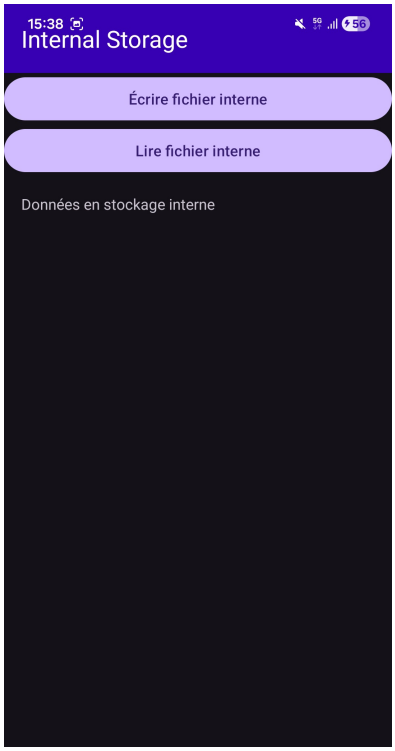
(b) (b) Saisie et sauvegarde des données



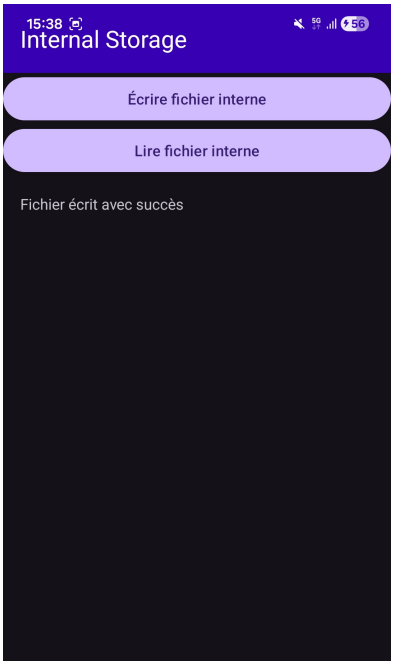
(c) (c) Lecture des données persistées

Figure 1: Utilisation de SharedPreferences pour stocker des données clé-valeur

10.2 Démonstration : App-specific Storage (Interne)



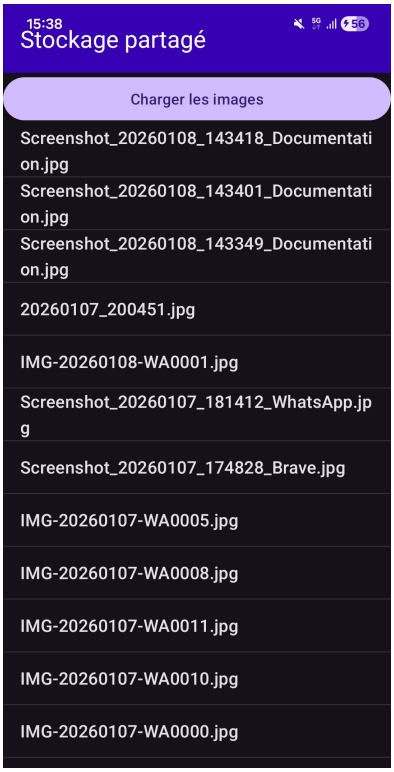
(a) (a) Écriture d'un fichier interne



(b) (b) Lecture du fichier stocké

Figure 2: Stockage interne spécifique à l'application

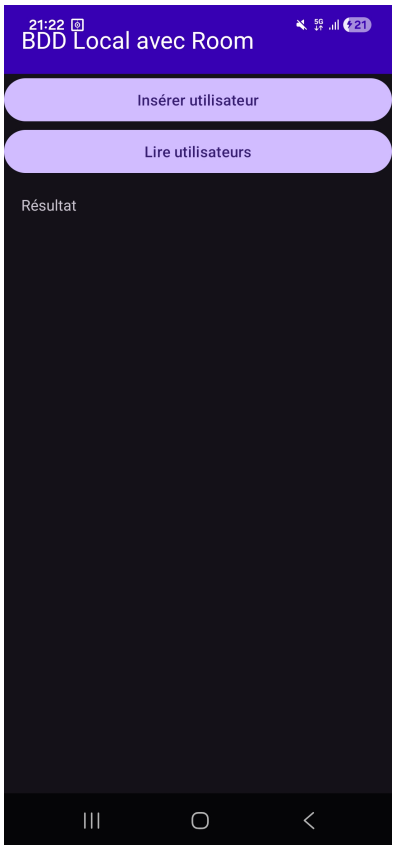
10.3 Démonstration : Accès aux images via MediaStore



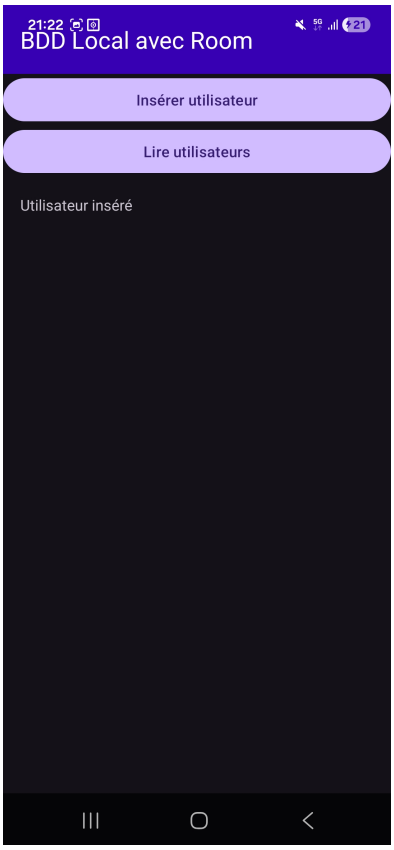
(a) (a) Bouton de chargement des images

Figure 3: Accès aux fichiers multimédias via l'API MediaStore

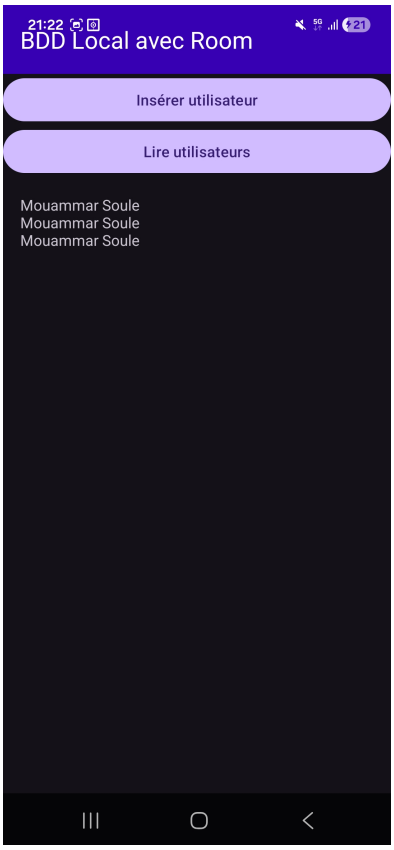
10.4 Démonstration : Base de données locale avec Room



(a) (a) Activity room



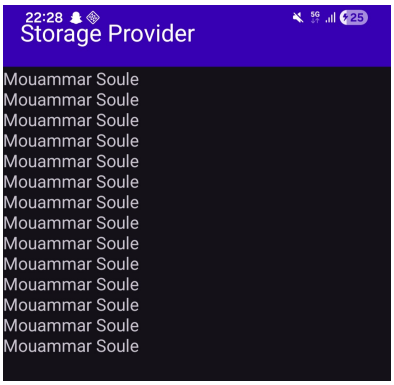
(b) (b) Insertion d'un utilisateur



(c) (c) Affichage des données persistées

Figure 4: Utilisation de Room pour stocker des données structurées

10.5 Démonstration : Content Provider



(a) (a) Données insérées via Room

Figure 5: Partage de données via un Content Provider